CATEGORY A

Coding Assignment

Data Structures · Arrays, Hashing, Two Pointers, Sliding Window, Prefix Sums, Binary Search

Problem 1: Mall Footfall Range Report

TASK

A shopping mall records the number of visitors for each hour of the day in an array. The operations team repeatedly asks for the total footfall between two hours, inclusive of both ends. On a busy day the team may send thousands of such range queries against the same day's data.

Implement a function `footfallReport(visitors, queries)` that returns a list containing the total visitors for each query `(start, end)`, in the same order as the queries. Your solution must answer every query without re-adding the values inside the range each time.

CONSTRAINTS

- 1 <= length of visitors <= 10^5
- 1 <= number of queries <= 10^5
- 0 <= start <= end < length of visitors

EXAMPLE

```
visitors = [12, 7, 3, 9, 15, 4, 8]
queries  = [(0, 2), (2, 5), (4, 6), (3, 3)]

Expected Output: [22, 31, 27, 9]
```

COMPLEXITY REQUIREMENTS

State the time complexity of answering all q queries with a naive approach, and then the time and additional space complexity of your optimized solution, including any one-time preprocessing. Explain when the preprocessing cost is worth paying.

HINT

> *Precompute a running total so that the sum of any range can be obtained from just two stored values. Be careful with the boundary when start is 0.*

Problem 2: Longest Budget-Friendly Streak

TASK

A college club tracks the daily cost of snacks for its study sessions in an array of non-negative integers. The treasurer wants to find the longest run of consecutive days whose total cost stays within a given budget.

Implement a function `longestStreak(costs, budget)` that returns a tuple `(length, startIndex)` for the longest contiguous run whose sum is less than or equal to `budget`. If several runs have the same maximum length, return the one that starts first. If no single day fits within the budget, return `(0, -1)`.

CONSTRAINTS

- 1 <= length of costs <= 10^5
- 0 <= costs[i] <= 10^4

- 0 <= budget <= 10^9

EXAMPLES

Sample 1

```
costs = [4, 2, 1, 7, 3, 1, 2, 1, 5], budget = 8

Expected Output: (4, 4)   // days 4 to 7: 3 + 1 + 2 + 1 = 7
```

Sample 2

```
costs = [9, 10], budget = 8

Expected Output: (0, -1)
```

COMPLEXITY REQUIREMENTS

State the time complexity of checking every possible run, and then the time and space complexity of your final solution. Explain why the fact that costs are non-negative is essential to your approach.

HINT

> *Unlike a fixed-size window, this window grows and shrinks. Extend the right end one day at a time, and move the left end forward only while the total exceeds the budget.*

Problem 3: Net-Balance Period Counter

TASK

A wallet app stores a user's daily net transactions as integers, where positive values are credits and negative values are debits. The audit team wants to know how many contiguous periods (runs of one or more consecutive days) have a net total exactly equal to a target amount k.

Implement a function `countPeriods(transactions, k)` that returns the number of contiguous subarrays whose sum equals `k`. Because the array can contain negative values, a technique that only shrinks or grows a window will not work correctly.

CONSTRAINTS

- 1 <= length of transactions <= 10^5
- -10^4 <= transactions[i] <= 10^4
- -10^9 <= k <= 10^9

EXAMPLES

Sample 1

```
transactions = [3, 4, -7, 1, 3, 3, 1, -4], k = 7

Expected Output: 4
// [3, 4], [3, 4, -7, 1, 3, 3], [1, 3, 3], [3, 3, 1]
```

Sample 2

```
transactions = [1, 2, 3], k = 10

Expected Output: 0
```

COMPLEXITY REQUIREMENTS

State the time complexity of a brute-force solution that sums every subarray, and then the time and additional space complexity of your optimized solution.

HINT

> *If the running total up to day j is S, then a period ending at day j sums to k exactly when an earlier running total was S − k. Count how often each running total has appeared so far.*

Problem 4: Exam Score Band Counter

TASK

A university stores the marks of all students in an exam as an array sorted in ascending order; marks can repeat. Faculty members frequently ask how many students scored within a band [low, high], inclusive of both ends.

Implement a function `countInBand(scores, low, high)` that returns the number of scores `s` such that `low <= s <= high`. The array is large and queried very often, so your solution must not scan the scores one by one.

CONSTRAINTS

- 1 <= length of scores <= 10^6
- scores is sorted in ascending order
- 0 <= low <= high <= 100

EXAMPLES

Sample 1

```
scores = [35, 42, 42, 50, 58, 58, 58, 63, 71, 88], low = 42, high = 58

Expected Output: 6
```

Sample 2

```
scores = [35, 42, 42, 50, 58, 58, 58, 63, 71, 88], low = 90, high = 100

Expected Output: 0
```

COMPLEXITY REQUIREMENTS

State the time and space complexity of a linear scan and of your final solution. Explain how duplicates at the edges of the band affect the way your search must be written.

HINT

> *A standard binary search stops at any matching element. Modify it to find the first position with a score >= low and the first position with a score > high; the answer is the difference.*

Problem 5: Spiral Stock Audit Route

TASK

An inventory robot audits a rectangular warehouse whose bins are stored as a 2D grid of bin IDs. To save travel time, the robot starts at the top-left bin and moves in a clockwise spiral: along the top row, down the right column, back along the bottom row, up the left column, and then continues inward until every bin has been visited exactly once.

Implement a function `auditRoute(grid)` that returns the list of bin IDs in the order the robot visits them. The grid may be rectangular (the number of rows and columns can differ).

CONSTRAINTS

- 1 <= rows, columns <= 100
- Every bin must appear exactly once in the output

EXAMPLE

```
grid = [[1,  2,  3,  4],
        [5,  6,  7,  8],
        [9, 10, 11, 12]]

Expected Output: [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]
```

COMPLEXITY REQUIREMENTS

State the time complexity in terms of m (rows) and n (columns), and the additional space complexity excluding the output list.

HINT

> *Maintain four boundaries (top, bottom, left, right) and shrink one after each side is traversed. Check that the boundaries have not crossed before walking the bottom row and the left column, or a single remaining row or column will be visited twice.*